Pseudocode for Exact Symmetry and Identifiability Analysis of Three-Layer Bottleneck ReLU Networks

Anonymous Authors

1 Scope and notation

We follow the notation of the paper. A three-layer architecture is

$$\mathcal{A} = (d, n_1, n_2, m),$$

and its parameter space is denoted by $\Theta_{\mathcal{A}}$. A parameter is

$$\theta = \left(W^{(\ell)}, b^{(\ell)}\right)_{\ell \in [3]} \in \Theta_{\mathcal{A}}.$$

The two ReLU layers and the final affine layer are

$$\phi^{(\ell)}(x) = [W^{(\ell)}x + b^{(\ell)}]_+, \quad \ell \in [2], \qquad T^{(3)}(y) = W^{(3)}y + b^{(3)},$$

and the realization is

$$f_\theta(x) = T^{(3)} \circ \phi^{(2)} \circ \phi^{(1)}(x).$$

We use $z^{(\ell,\theta)}$ and $a^{(\ell,\theta)}$ for the preactivation and activation maps, respectively, with $a^{(0,\theta)}(x) = x$.

The fiber and generic fiber are

$$\mathcal{S}(\theta) := \{\eta \in \Theta_{\mathcal{A}} : f_\eta = f_\theta\}, \qquad \widetilde{\mathcal{S}}(\theta) := \mathcal{S}(\theta) \cap \widetilde{\Theta}_{\mathcal{A}}.$$

The relation $\theta \sim \eta$ denotes equivalence under hidden-neuron permutations and positive rescalings. Thus θ is identifiable when $f_\eta = f_\theta$ implies $\eta \sim \theta$.

Throughout, $\mathcal{A} = (d, n_1, n_2, m)$ is fixed with $d \geq n_1 \geq 2$, and all comparisons, signs, normalizations, and matrix identities are exact. The algorithms accept zero entries. Every positive equivalence or non-identifiability certificate is valid for arbitrary exact parameters; completeness and identifiability are asserted only on $\widetilde{\Theta}_{\mathcal{A}}$.

Following Definition 7 of the paper, a second-layer neuron $i \in [n_2]$ *hides hyperplane* $j \in [n_1]$ if

$$W^{(2)}_{ij} > 0, \qquad W^{(2)}_{ik} \leq 0 \quad (k \neq j), \qquad b^{(2)}_i \leq 0.$$

The parameter θ hides hyperplane j if every $i \in [n_2]$ does. These weak inequalities include exact parameters with zeros. On $\widetilde{\Theta}_{\mathcal{A}}$, the relevant signs are strict.

2 Exact functional-equivalence certificate

Listing 1: Exact equivalence dispatcher

```
algorithm EquivalenceTest(θ, η ∈ Θ_A)
output: true if the certificate proves f_θ = f_η

θ̄ <- RemoveDeadSecondLayer(θ)
η̄ <- RemoveDeadSecondLayer(η)

```

```
if Width2(θ̄) = Width2(η̄) = 0 then
    return b̄^(3,θ) = b̄^(3,η)
else if exactly one reduced width is zero then
    return false
end if

j_θ <- UniqueWeakHidingIndex(θ̄)
j_η  <- UniqueWeakHidingIndex(η̄)

if exactly one of j_θ, j_η is unknown then
    return false
end if

if both j_θ, j_η are unknown then
    return NonHidingCertificate(θ̄, η̄)
else if Width2(θ̄) != Width2(η̄) then
    return false
else if Width2(θ̄) >= 2 then
    return MultiHidingCertificate(
        θ̄, η̄, j_θ, j_η)
else
    return WidthOneHidingCertificate(
        θ̄, η̄, j_θ, j_η)
end if
```

For a non-hiding parameter, the induced one-hidden-layer map uses the paper's notation

$$g_{\theta'} : \mathbb{R}^{n_1}_{\geq 0} \to \mathbb{R}^m, \qquad g_{\theta'}(y) = W^{(3)}[W^{(2)}y + b^{(2)}]_+ + b^{(3)},$$

where $\theta' = (W^{(2)}, b^{(2)}, W^{(3)}, b^{(3)})$.

Listing 2: Non-hiding certificate

```
algorithm NonHidingCertificate(θ, η)
(θ°, η°) <-
    CanonicalizeFirstLayerModuloPositiveScalingAndPermutation(
        θ, η)

if [W^(1,θ°), b^(1,θ°)] ≠ [W^(1,η°), b^(1,η°)] then
    return false
end if

(H_θ, A_R,θ, b_R,θ, c_θ) <-
    HyperplaneRepresentation(g_θ')
(H_η, A_R,η, b_R,η, c_η) <-
    HyperplaneRepresentation(g_η')

require H_θ = H_η
require c_θ(i) = c_η(i) on every H_i ∈ H_θ
require A_R,θ = A_R,η and
        b_R,θ = b_R,η on one common region R
return true exactly when all requirements hold
```

Listing 3: Hiding certificates

```
algorithm MultiHidingCertificate(θ, η, j_θ, j_η)
normalize the hidden first-layer direction
normalize and order all non-hidden affine hyperplanes
```

```
normalize every active second-layer row by its positive hidden coefficient
order second-layer rows canonically

require equal normalized first-layer directions
require equal non-hidden first-layer biases
require equal normalized W^(2), W^(3), b^(3)

t <- b^(1,η)_{j_η} − b^(1,θ)_{j_θ}
require b^(2,η) = b^(2,θ) − t W^(2,θ)_{:,j_θ}
require t ≥ max_{i∈[n_2]} b^(2,θ)_i / W^(2,θ)_{i j_θ}
return true exactly when all requirements hold

algorithm WidthOneHidingCertificate(θ, η, j_θ, j_η)
if MultiHidingCertificate(θ, η, j_θ, j_η) = true then
    return true
end if

normalize the nonzero active row and output direction
match non-hidden first-layer hyperplanes up to unoriented scaling
recover the set of coordinates whose orientations are reversed
recover I ⊆ [n_1] \ {j} and apply M_I ∈ G_j
require equality after the recovered sign flip, except for translation
recover t and require η ∈ T_j(M_I · θ)
return true exactly when all requirements hold
```

Here

$$T_j(\theta) = \left\{ \left(W^{(1)},\ b^{(1)} + t e_j,\ W^{(2)},\ b^{(2)} - t W^{(2)}_{:,j},\ W^{(3)},\ b^{(3)} \right) : t \geq \max_{i \in [n_2]} \frac{b^{(2)}_i}{W^{(2)}_{ij}} \right\}$$

is the paper's translation family, and for $n_2 = 1$

$$M_I = I_{n_1} - \sum_{k \in I} e_j e_k^\top - 2 \sum_{k \in I} e_k e_k^\top, \qquad \mathcal{G}_j = \{ M_I : I \subseteq [n_1] \setminus \{j\} \}.$$

3 Symmetry and identifiability classification

Listing 4: Symmetry and identifiability classification

```
algorithm ClassifySymmetries(θ)
input: θ ∈ Θ_A
output: certified non-identifiable, generic-ID candidate,
        or undetermined

dead <- {i ∈ [n_2] :
         every entry of [W^(2)_{i,:}, b^(2)_i] is nonpositive}
linear <- {i ∈ [n_2] :
           every entry of [W^(2)_{i,:}, b^(2)_i] is nonnegative}
hiding <- {j ∈ [n_1] : θ hides hyperplane j}

if dead ≠ ∅ or linear ≠ ∅
   or hiding ≠ ∅ then
    return certified non-identifiable
else if SomeEntryIsZero([W^(2), b^(2)]) then
    return undetermined
else
```

```
    return generic-ID candidate
end if
```

On $\widetilde{\Theta}_{\mathcal{A}}$, this is Theorem 13: non-identifiability is equivalent to a strictly negative row, a strictly positive row, or a common strict hiding column. With zero entries, the weak conditions still give explicit symmetries; otherwise the generic conclusion is left undetermined.

4 Generic fiber dimension

For the induced map $g_{\theta'} : \mathbb{R}^{n_1}_{\geq 0} \to \mathbb{R}^m$, let $\mathcal{H} = \{H_1, \ldots, H_k\}$ be its breakpoint-hyperplane arrangement and let $J = [n] \setminus [k]$ index the linear neurons, as in Proposition 5. Thus $r := |J| = n - k$. The *essential dimension* is $\dim(V_S)$, equivalently the dimension modulo $\sim$. The *full dimension* restores the $n_1 + n_2$ positive-scaling directions.

Listing 5: Dimension of the generic fiber modulo (*@∼@*)

```
algorithm GenericFiberDimension(θ)
input: generic θ; in the non-hiding case, one of Theorem 6's cases
(θ̄, dead) <- RemoveDeadSecondLayer(θ)
q <- number of removed dead neurons

if Width2(θ̄) = 0 then
    return dim(Θ_A) − m − (n_1 + n_2)
end if

if θ̄ hides hyperplane j then
    active_dimension <- dim(T_j(θ̄)/∼) = 1
else
    form the induced map g_θ̄' : R^{n_1}_{≥0} → R^m
    k <- number of nonlinear neurons of g_θ̄'
    J <- linear-neuron set [n] \ [k]
    r <- |J| = n − k

    if r < min(n_1, m) then
        active_dimension <- dim(V_{S_θ̄'}) = r^2
    else if r = n_1 ≤ m then
        active_dimension <- dim(V_S) = r^2
    else if r = m < n_1 then
        active_dimension <- dim(V_S) = r^2
                            for every feasible orientation S
    else if r = m + 1 ≤ n_1 then
        active_dimension <- dim(V_S) = m^2 + m + n_1
    end if
end if

return active_dimension + q(n_1 + m)
```

The cases for V_S are those of Theorem 6 applied to the induced shallow map. The full fiber dimension is obtained by restoring the $n_1 + n_2$ positive-scaling directions. This dimension statement is made under the generic assumptions of the theorem. ty probability.